

Intro to Deep Learning

Lecture 15: From GANs To Diffusion

Stanislav Don

DS212 - Intro to Deep Learning

Today

By the end of the lecture, you should be able to answer:

1. What makes generative modeling different from classification?
2. What is the core idea of a GAN?
3. Why can GAN training be difficult?
4. What is the core idea of diffusion?
5. How does text conditioning guide image generation?
6. Why did diffusion become so important for modern image generation?

Lecture rhythm: generation -> GANs -> noise -> denoising -> text conditioning -> limits

Recap From The Course

We have already seen most of the ingredients:

- CNNs learn visual features
- ResNets make deep image models easier to train
- attention mixes information by content
- Transformers process token sequences
- CLIP connects images and text in one embedding space
- LLMs generate text token by token

Today:

how do neural networks generate images?

Part 1

What does it mean to generate?

Classification vs Generation

Classification:

```
image -> model -> class label
```

Generation:

```
randomness / prompt -> model -> new sample
```

Classification chooses among known labels.

Generation creates a new example that should look like it came from the data.

A Different Kind Of Learning

A classifier learns a decision boundary:

is this class A or class B?

A generative model tries to learn a data distribution:

what do realistic examples look like?

For images, this means learning patterns of:

- shapes
- textures
- object parts
- styles

Not Memorization

A useful generative model should not simply copy training examples.

It should learn enough structure to create new examples:

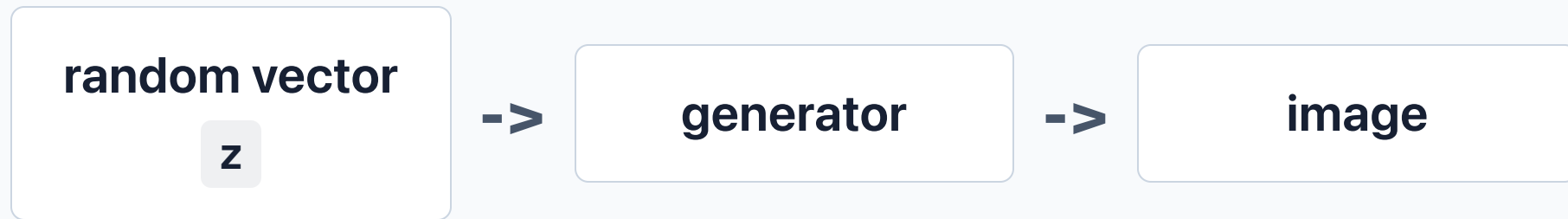
```
new digit  
new food photo  
new face-like image  
new scene matching a prompt
```

The hard part:

```
real images live in a very complicated space
```

Latent Variable Intuition

Many generative models start from a hidden random vector:



The random vector controls the generated sample.

Different random starts can produce different outputs.

Mini-Check

Suppose a model generates 10,000 images.

All of them are sharp.

All of them are the same kind of image.

Is this a good generative model?

Answer

Not really.

It may produce high-quality samples, but it has poor **diversity**.

A good generative model should usually have:

- quality
- diversity
- controllability
- reasonable coverage of the data distribution

Part 2

GANs: the first big wow

GAN Big Idea

GAN means:

Generative Adversarial Network

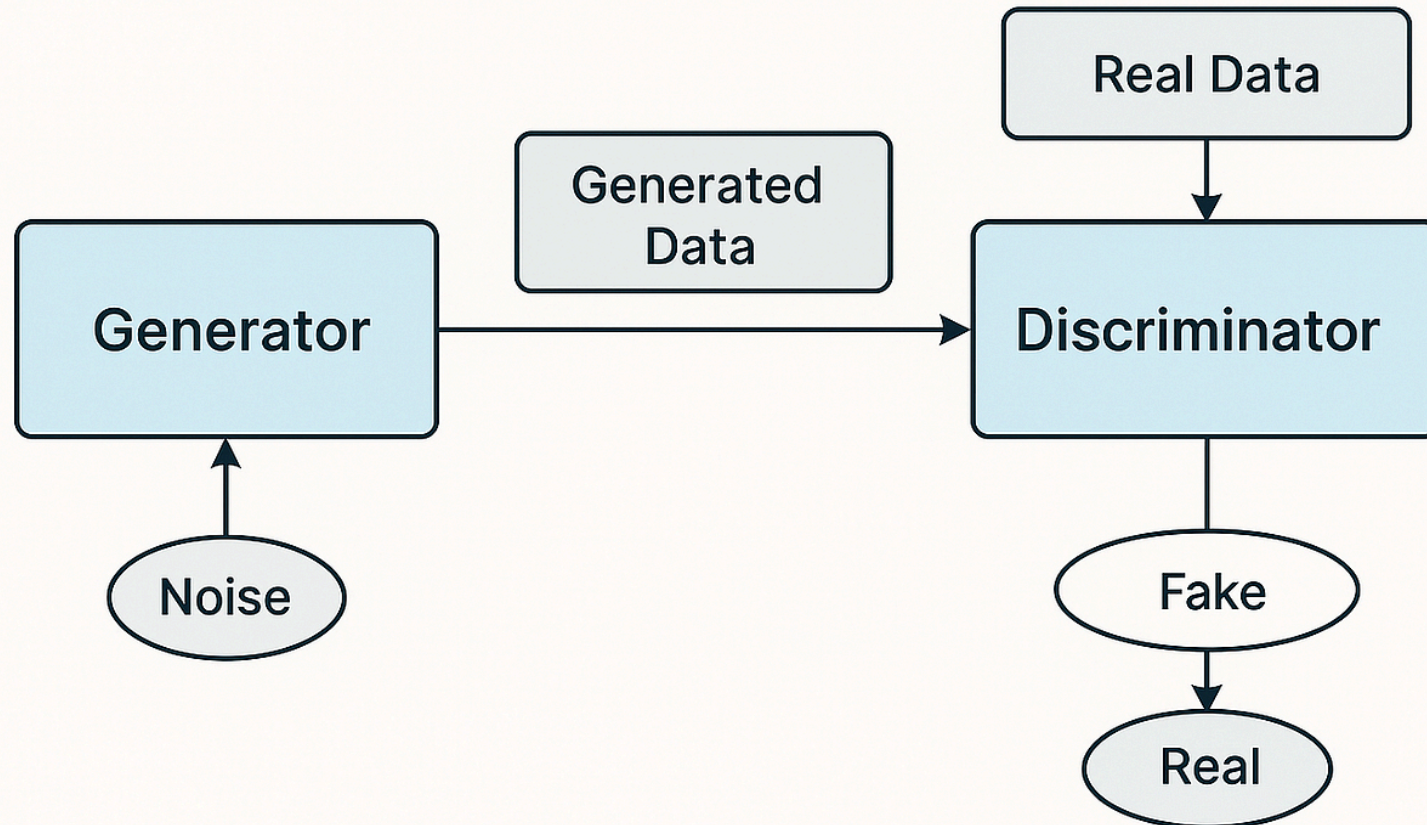
It trains two neural networks together:

- **generator**: creates fake examples
- **discriminator**: predicts real vs fake

The generator learns by trying to fool the discriminator.

GAN Architecture

GAN Architecture



GAN Training As A Game

The discriminator wants:

```
real images -> real  
fake images -> fake
```

The generator wants:

```
fake images -> real
```

So training is adversarial:

```
one network improves by challenging the other
```

A Simple Analogy

Generator

Counterfeiter.

Learns to produce images that look real enough to pass inspection.

Discriminator

Detective.

Learns to distinguish real images from generated images.

The game is useful only if both sides keep improving.

Why GANs Were Exciting

GANs showed that neural networks could generate:

- sharp images
- realistic faces
- plausible textures
- new samples very quickly after training

After training, sampling can be one forward pass:

```
z -> generator -> image
```

This made GANs a huge milestone in generative AI.

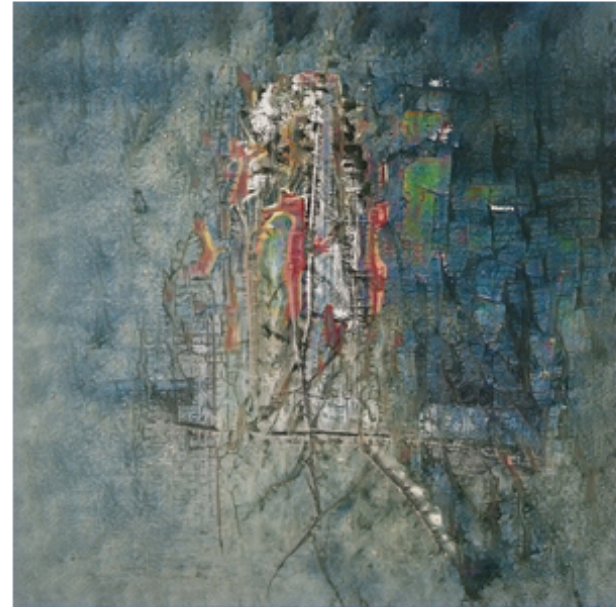
Real GAN Outputs



StyleGAN2 face



StyleGAN2 cat



StyleGAN2 artwork

Real generated examples, not redrawn diagrams

GANs became famous because samples could be sharp and visually convincing.

Why GANs Are Hard

GAN training is not just ordinary supervised learning.

Common problems:

- training instability
- generator/discriminator imbalance
- mode collapse
- difficult evaluation
- weak controllability

The loss curves can be hard to interpret.

Sometimes one model wins too early.

Mode Collapse

Mode collapse means:

the generator finds a small set of outputs
that fool the discriminator
and keeps producing them

Example:

many sharp generated digits
but almost all of them are 7s

Quality can look high while diversity is poor.

GANs vs The Control Problem

For many applications, we want to specify the output:

```
"a red car on a snowy road"  
"a Thai dessert on a wooden table"  
"same image, but change the background"
```

Classic GANs can be conditioned.

But precise text control and editing were not their strongest natural form.

Transition

GANs ask:

Can I fool a discriminator?

Diffusion asks:

Can I undo noise?

Part 3

Diffusion: learn to denoise

Diffusion In One Sentence

A diffusion model learns to generate images by learning how to **denoise** them.

Training:

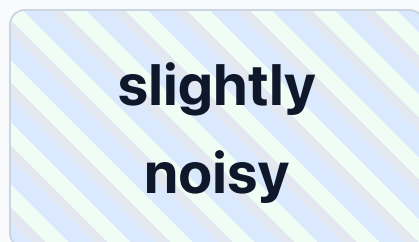
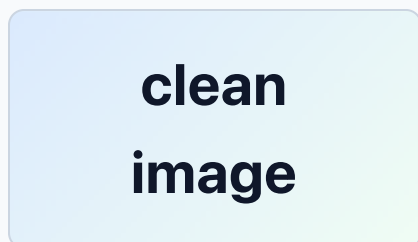
```
clean image -> add noise -> model predicts the noise
```

Sampling:

```
random noise -> repeatedly denoise -> generated image
```


Forward Process

The forward process gradually destroys an image.

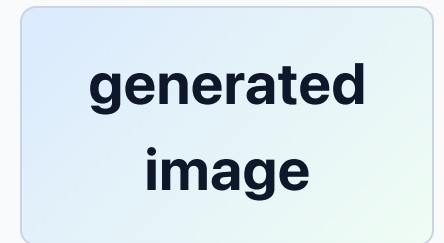
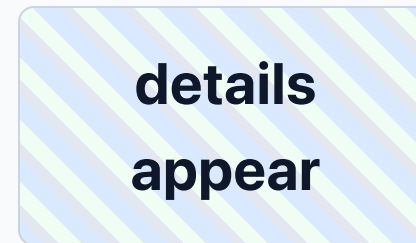
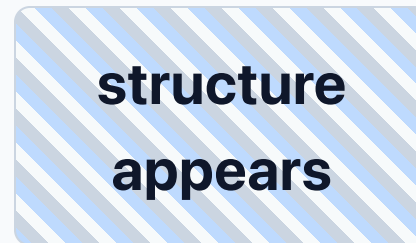


This part is fixed.

No neural network is needed to add noise.

Reverse Process

Generation runs the process backward.



The neural network learns this reverse direction.

What The Model Learns

At training time, we know:

- the original image
- the noise we added
- the timestep/noise level
- the noisy image

So we train the model:

```
model(noisy image, timestep) -> predicted noise
```

Then compare predicted noise with the true noise.

Training Objective

One common simplified DDPM objective:

$$L = \|\epsilon - \epsilon_{\theta}(x_t, t)\|^2$$

Where:

- x_t is the noisy image at timestep t
- ϵ is the true noise
- ϵ_{θ} is the model's predicted noise

This is just supervised learning on a denoising task.

Why This Feels Familiar

The architecture is new.

The workflow is not:

```
data -> model -> prediction -> loss -> gradients -> update
```

The model does not predict a class label.

It predicts an image-shaped noise tensor.

Timesteps

The same denoising model handles many noise levels.

```
t small  -> image is only slightly noisy  
t large  -> image is almost pure noise
```

So the model receives timestep information:

```
model(x_t, t)
```

Otherwise it would not know how aggressive the denoising should be.

Why Predict Noise?

If the model can predict what noise was added, it can remove part of it.

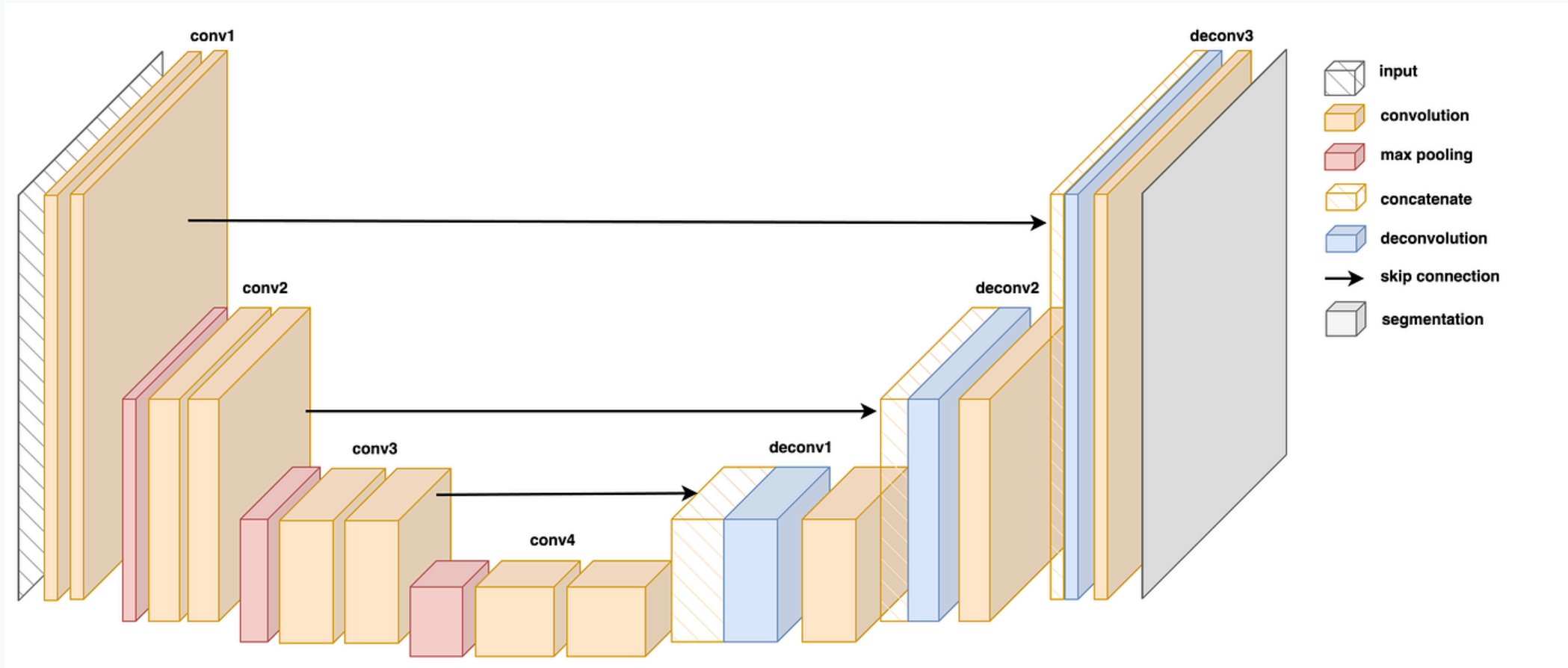
One denoising step:

noisy image – predicted noise → slightly cleaner image

Many steps:

noise → structure → object → detail → final image

The Model Inside: Often A U-Net



How U-Net Helps Denoising

For image diffusion, the denoising model often uses a U-Net-like architecture.

- downsample to understand global structure
- process features at multiple resolutions
- upsample to reconstruct detail
- use skip connections to preserve local information

The goal is image-shaped prediction:

```
noisy image + timestep -> predicted noise
```

"Deconv" often means transposed convolution: a learnable upsampling operation.

Sampling Algorithm

High-level pseudocode:

```
x = random_noise()

for t in reversed(timesteps):
    predicted_noise = model(x, t)
    x = remove_a_little_noise(x, predicted_noise, t)

image = x
```

Generation is an iterative refinement process.

Seed Intuition

The seed controls the initial noise.

```
same prompt + same seed -> similar/same image  
same prompt + different seed -> different image
```

So a seed is not magic.

It is a reproducibility handle for the random starting point.

Mini-Check

Why can diffusion sampling be slower than GAN sampling?

Think about how the final image is produced.

Answer

A classic GAN generator can create an image in one model pass:

```
z -> image
```

A diffusion model usually needs many model passes:

```
noise -> denoise -> denoise -> ... -> image
```

Fewer denoising steps usually means faster and cheaper generation.

That is why faster samplers matter in practice.

Part 4

From image generation to text-to-image

Unconditional vs Conditional Generation

Unconditional generation:

```
noise -> image
```

Conditional generation:

```
noise + condition -> image
```

The condition can be:

- text prompt
- image
- depth map
- sketch

Text-To-Image Pipeline

Text-to-image diffusion adds a text condition.

```
prompt  
  -> text encoder  
  -> text embedding
```

```
random noise + text embedding  
  -> denoising model  
  -> generated image
```

The text embedding conditions each denoising step.

Where Attention Returns

In many text-to-image systems, image features attend to text features.

Prompt:

"a red car on a snowy road"

Different image regions can use different words:

- red
- car
- snowy
- road

This is why Lecture 08 and Lecture 14 matter here.

Cross-Attention Intuition

Self-attention:

tokens look at other tokens

Cross-attention:

image features look at text tokens

So the image-generation process can ask:

which words should guide this part of the image?

Classifier-Free Guidance

Guidance controls how strongly the prompt pulls the generation.

Intuition:

```
prediction with prompt  
minus  
prediction without prompt  
= prompt direction
```

The guidance scale amplifies this direction.

Low guidance:

```
more freedom, less prompt-following
```

High guidance:

Negative Prompts

A negative prompt describes visual directions to avoid.

Example:

```
prompt: "a clean product photo of a coffee cup"  
negative: "blurry, distorted, low quality"
```

This is not logical reasoning.

It is steering the denoising process away from certain text conditions.

Why use it?

- reduce common artifacts
- discourage unwanted objects
- push away from low-quality visual patterns

Editing With Diffusion

Diffusion also supports editing workflows:

- image-to-image
- inpainting
- outpainting
- super-resolution
- style transfer-like edits

Core idea:

```
keep part of the signal  
regenerate part of the signal
```

We save these for the practical part.

Part 5

Why diffusion became dominant

GANs vs Diffusion

Feature	GANs	Diffusion
Training idea	adversarial game	denoising objective
Sampling	often fast	often slower
Training stability	difficult	usually easier
Diversity	can collapse	usually strong
Text control	possible, harder	very strong
Editing workflows	possible	very natural

Why Diffusion Worked So Well

Diffusion models became important because they combine:

- stable supervised-style training objective
- high image quality
- strong diversity
- good text conditioning
- natural editing workflows
- scalable architectures and data

The result:

powerful generation + useful control knobs

What Diffusion Still Struggles With

Common limitations:

- slow sampling compared with one-pass models
- high compute cost
- prompt sensitivity
- counting and spatial relationships
- text inside images
- consistency across many frames
- bias from training data

The outputs can look confident even when they are wrong.

Ethics And Society

Generative models raise real questions:

- copyright and training data
- fake media and provenance
- consent and likeness
- bias and representation
- economic effects on creative work
- misuse vs beneficial applications

Technical skill should come with judgment.

Future Directions

Active directions include:

- faster image generation
- video diffusion
- 3D generation
- personalized generation
- controllable editing
- multimodal assistants
- world models for robotics and simulation

Image generation is becoming one component in larger AI systems.

Takeaways

1. Generative models learn to create new samples, not just label inputs.
2. GANs use an adversarial generator-discriminator game.
3. GANs were historically important, but training can be unstable.
4. Diffusion learns to reverse a noise process.
5. Text-to-image models condition denoising on prompt embeddings.
6. Diffusion is powerful, controllable, and imperfect.

End Of The Course

Thank you

You now have the vocabulary to keep learning:

data, models, losses, gradients, representations, attention, adaptation, generation.

Stay in touch:

<https://www.linkedin.com/in/stanislav-don-751542244/>